
NEW NECOYOAD · PROJECT PLAN FOR A MODERN REBUILD

New Necoyoad

v12

*A digital web agency platform built on Laravel 11,
preserving the widget engine and key architectural patterns
from the original*

stack: Laravel 11 + PHP 8.3 + Filament 3 + Livewire 3
+ Alpine.js + MySQL 8 + Redis + Vite

preserved: 10 patterns (widget engine, declarative CRUD,
polymorphic spine, Hooks/Events, multi-store, etc.)

plan: 7 phases, 80 tasks, 22 weeks (1 dev) / 3 months (2 devs)

pages: 16

Architecture Design Report

Based on 11 volumes (248 pages) of verified analysis
github.com/yosietserga/necoyoad

PRESCRIPTIVE · BUILD PLAN · THE BUILD CAN BEGIN

New Necoyoad — Project Plan for a Modern Rebuild (v12)

*A digital web agency platform built on Laravel 11,
preserving the widget engine and key architectural patterns from the original*

Abstract

This is the final volume of the Necoyoad architecture blueprint. It is a complete project plan for building a *new* Necoyoad — a modern digital web agency platform — using the latest PHP/Laravel stack while preserving the most valuable architectural patterns from the original codebase (analysed across v1–v11, 248 pages).

The plan defines: (1) the technology stack (Laravel 11, PHP 8.3, Filament 3 admin, Livewire 3 storefront, Alpine.js, MySQL 8 InnoDB, Redis, Vite); (2) the 10 architectural patterns preserved from the original (widget engine, declarative admin CRUD, polymorphic object spine, dual Hooks/Events, multi-store/multi-language, per-entity template override, banner plugin discriminator, campaign pipeline, visual theme editor, ntPlugins JS registry); (3) the database schema (50 tables, InnoDB, JSON columns, proper FKs); (4) the directory structure; (5) the 7-phase implementation plan with 80 tasks, effort estimates, and milestones; and (6) the widget engine API contract — the formal specification that ensures the new platform has the same flexibility as the original.

The target: a platform where a non-technical user can compose rich, interactive pages by dragging widgets into positions, a developer can create a new widget in minutes, and the system supports multi-store, multi-language, per-entity overrides, and a visual theme editor — all on a modern, testable, secure foundation.

Keywords: Necoyoad, Laravel 11, PHP 8.3, Filament 3, Livewire 3, Alpine.js, widget engine, digital agency, multi-store, multi-language, project plan, architecture design.

Contents

1	Introduction	3
1.1	From Analysis to Action	3
1.2	Goal	3
1.3	What This Document Is	3
2	Technology Stack	3
3	The 10 Preserved Patterns	4
3.1	Pattern 1: The Widget Engine (Core Feature)	4
3.2	Pattern 2: Declarative Admin CRUD	4
3.3	Pattern 3: Polymorphic Object Spine	5
3.4	Pattern 4: Dual Hooks/Events System	5
3.5	Patterns 5–10: Summary	5
4	Database Schema	5
4.1	Design Principles	6
4.2	Core Tables (35 tables, down from 87)	6
5	Directory Structure	7
6	Implementation Plan	9
6.1	Overview: 7 Phases, 80 Tasks, 8 Months	9
6.2	Phase 1: Foundation (Weeks 1–3)	9
6.3	Phase 2: Widget Engine (Weeks 4–7) — The Critical Phase	10
6.4	Phase 3: Admin Panel (Weeks 8–10)	11
6.5	Phase 4: Storefront (Weeks 11–14)	11
6.6	Phase 5: CMS + Banners (Weeks 15–16)	11
6.7	Phase 6: Marketing (Weeks 17–19)	12
6.8	Phase 7: Polish + Deploy (Weeks 20–22)	12
7	The Widget Engine API Contract	13
8	Creating a New Widget Module	13
9	What is Discarded	15
10	Milestones	15
11	Conclusion	15
11.1	The Vision	15
11.2	The Widget Engine Survives	15
11.3	The Timeline is Achievable	16
11.4	Final Word	16

1 Introduction

1.1 From Analysis to Action

Volumes v1–v11 analysed the existing Necoyoad codebase exhaustively: 248 pages of verified architecture documentation covering every subsystem, plus a technical assessment (v10) and a widget engine preservation strategy (v11). This volume turns that analysis into a build plan.

1.2 Goal

Build a **new** Necoyoad — a digital web agency platform — that:

- Uses the latest PHP/Laravel stack (Laravel 11, PHP 8.3).
- Preserves the 10 most valuable architectural patterns from the original.
- Discards the 15 technical debts identified in v10.
- Supports multi-store, multi-language, and multi-domain out of the box.
- Has the widget engine as its core feature — the same “drag a widget into a position, configure it, see it on the page” experience.
- Is testable, secure, and deployable via Docker.

1.3 What This Document Is

This is not a tutorial or a code dump. It is an *architecture design and project plan* — the document a senior architect hands to a development team to say “build this”. It defines the stack, the patterns, the schema, the directory structure, the implementation phases, and the acceptance criteria.

2 Technology Stack

Table 1: Technology stack for New Necoyoad.

Layer	Technology	Rationale
Language	PHP 8.3	Typed properties, readonly, enums, fibers.
Framework	Laravel 11	Mature, expressive, huge ecosystem.
Admin panel	Filament 3	Declarative resource CRUD (maps to AdminController).
Storefront	Livewire 3 + Alpine.js	Reactive UI without SPA complexity.
Templating	Blade	Native to Laravel, supports dynamic components + stacks.
Database	MySQL 8 (InnoDB)	JSON columns, full-text search, transactions, FKs.
Cache	Redis	Session, cache, queue, widget tree cache.
Queue	Laravel Horizon	Campaign sending, bounce processing, async tasks.
Search	Meilisearch (optional)	Full-text product/post search.
Assets	Vite 5 + Tailwind CSS 4	Modern bundling, JIT CSS, HMR.
Auth	Laravel Sanctum + Fortify	API tokens, session auth, 2FA.
Testing	Pest 2 + Dusk	Unit, integration, browser tests.
CI/CD	GitHub Actions	Automated tests + deploy on push.
Deployment	Docker + FrankenPHP	Modern PHP application server.

3 The 10 Preserved Patterns

3.1 Pattern 1: The Widget Engine (Core Feature)

Widget Engine — the crown jewel preserved

The widget engine is the platform's defining feature. It must provide:

1. **WidgetService:** queries the widget tree from the DB (JSON columns, not LIKE). Filters by store, language, device, route, landing page, object type, object id.
2. **Widget View Composer:** a Laravel View Composer that runs before every page render, calls WidgetService, populates `$widgets[$position]` with the tree.
3. **WidgetComponent base:** a Blade Component base class that auto-loads assets (via AssetManifest) and provides the `data()` method (the injection seam, replacing `module:settings`).
4. **Dynamic composition:** the page template iterates `$widgets` and renders `<x-dynamic-component>`.
5. **Manual composition:** templates use `@stack` + `@push` to hardcode widget positions.
6. **Hybrid composition:** both in the same template.
7. **Per-entity override:** WidgetService accepts `objectType` + `objectId`, merges per-object widgets into the default tree.
8. **Per-item composition:** same, with `objectType = 'banner_item'` (or any item type).
9. **Visual editor hooks:** every widget wrapper emits `data-widget`, `data-position`, `data-row`, `data-column` attributes.
10. **Template override:** each entity can specify a template path (JSON column), resolved as: entity template → config default → hardcoded fallback.

3.2 Pattern 2: Declarative Admin CRUD

AdminController → Filament Resources

Filament 3 provides declarative resource CRUD out of the box. Each admin entity is a Filament Resource with:

- Form schema (replaces `$form_vars`).
- Table columns + filters (replaces `$filters`).
- Pages (list, create, edit) — all generated.
- Relations (repeater for banner items, etc.).
- Middleware for permission checks (replaces `$public_methods`).

A base `NecoyoadResource` class can provide shared functionality (breadcrumbs, activity logging, SEO tab, store assignment tab).

3.3 Pattern 3: Polymorphic Object Spine

Polymorphic spine → Eloquent Morph Relations

The five polymorphic tables become Eloquent traits:

- **HasDescriptions** trait: MorphMany to Description model, scoped by `language_id`.
- **HasProperties** trait: MorphMany to Property model, with `group` + `key` accessors.
- **HasStoreAssignment** trait: MorphToMany to Store model.
- **HasSeoUrl** trait: MorphMany to UrlAlias model, scoped by `language_id`.
- **HasCategories** trait: MorphToMany to Category model.

Any model that uses these traits gets localisation, EAV properties, store scoping, SEO URLs, and category assignment — automatically.

3.4 Pattern 4: Dual Hooks/Events System

Hooks + Events → Laravel Events + Pipeline

- **Events** (observer): Laravel's native `Event::dispatch()` + listeners. For notifications (dispatch, beforeLoad, afterLoad, registry:update).
- **Hooks** (filter/action with short-circuit): a custom `Filter` facade using Laravel's Pipeline pattern. `Filter::apply('render', $html)` chains the HTML through registered filters. `Filter::run('loadWidgets', $params)` can short-circuit by returning a value.

Both fire at the same points as the original: `render`, `fetch`, `loadWidgets`, `module:settings`, `csspath`, `redirect`, etc.

3.5 Patterns 5–10: Summary

Table 2: Patterns 5–10 preserved.

#	Pattern	Implementation
5	Multi-store	Tenancy scope on all models; <code>StoreContext</code> middleware resolves tenant from domain/path/subdomain.
6	Multi-language	6-level detection middleware; <code>HasDescriptions</code> trait provides per-language content.
7	Per-entity template override	JSON column <code>template</code> on each entity; resolved in the controller: entity → config → default.
8	Banner plugin discriminator	<code>jquery_plugin</code> column on <code>banners</code> table; dynamic Blade component: <code><x-sliders.\$plugin></code> .
9	Campaign pipeline	Laravel Queue jobs: <code>SendCampaignEmail</code> , <code>ProcessBounce</code> ; Laravel Mail with SMTP/OAuth2; tracking via controller routes.
10	ntPlugins JS registry	Alpine.js <code>\$dispatch</code> or Livewire events; each slider template pushes config to an Alpine store.

4 Database Schema

4.1 Design Principles

- All tables InnoDB with proper foreign keys.
- Widget settings stored as JSON columns (not serialised PHP).
- Per-store settings as JSON column on `stores` table.
- No LIKE substring queries — use JSON path expressions on JSON columns.
- UUIDs for public-facing IDs (products, posts, orders); ints for internal.
- Soft deletes on all content tables.
- `created_at`, `updated_at`, `deleted_at` on all tables.

4.2 Core Tables (35 tables, down from 87)

Table 3: Core database tables for New Necoyoad.

Table	Purpose
Multi-tenancy & localisation	
<code>stores</code>	Store definitions (domain, folder, settings JSON).
<code>languages</code>	Active languages.
<code>currencies</code>	Currencies + exchange rates.
<code>countries</code>	ISO countries.
<code>zones</code>	States/provinces.
<code>geo_zones</code>	Tax rule groupings.
Polymorphic spine	
<code>descriptions</code>	Localised content (morph: <code>object_id</code> + <code>object_type</code> + <code>language_id</code>).
<code>properties</code>	EAV metadata (morph: <code>object_id</code> + <code>object_type</code> + <code>group</code> + <code>key</code>).
<code>url_aliases</code>	SEO slugs (morph: <code>object_id</code> + <code>object_type</code> + <code>language_id</code>).
<code>categorizables</code>	Polymorphic category assignment (morph: <code>categorizable_id</code> + <code>categorizable_type</code>).
<code>store_assignments</code>	Polymorphic store scoping (morph: <code>assignable_id</code> + <code>assignable_type</code> + <code>store_id</code>).
Catalog	
<code>products</code>	Product master (SKU, price, stock, weight, dimensions).
<code>product_options</code>	Orderable variants (colour, size).
<code>product_option_values</code>	Variant values with price deltas.
<code>product_images</code>	Additional images.
<code>product_discounts</code>	Quantity-break pricing per customer group.
<code>product_specials</code>	Promotional pricing per customer group.
<code>categories</code>	Category tree (nested set or adjacency list).
<code>manufacturers</code>	Brand directory.
<code>reviews</code>	Polymorphic reviews with ratings + likes.
CMS	
<code>posts</code>	Blog posts + static pages (discriminated by <code>type</code>).
<code>menus</code>	Named menus per store/position.
<code>menu_links</code>	Menu link tree (<code>parent_id</code> , <code>link</code> , <code>sort_order</code>).
<code>banners</code>	Banner definitions (<code>jquery_plugin</code> , settings JSON).
<code>banner_items</code>	Slides (<code>image</code> , <code>link</code> , <code>sort_order</code>).
Widget engine	
<code>widget_rows</code>	Layout rows (<code>store_id</code> , <code>position</code> , <code>sort_order</code> , settings JSON).
<code>widget_columns</code>	Columns within rows (<code>row_id</code> , <code>grid config</code> , settings JSON).

Table	Purpose
widgets	Widget instances (column_id, module, name, settings JSON, sort_order).
widget_landing_pages	Widget-to-route bindings (widget_id, landing_page, object_type).
Commerce	
customers	Customer accounts.
customer_groups	Pricing/permission tiers.
addresses	Customer address book.
orders	Order header (snapshot of customer + totals).
order_items	Order line items (snapshot of product + options).
order_totals	Computed total lines (sub-total, tax, shipping, etc.).
order_history	Status change log.
coupons	Discount codes.
Marketing	
contacts	CRM contacts.
contact_lists	Named contact lists.
contact_list_subscriptions	Many-to-many: contact ↔ list.
newsletters	Reusable email bodies.
campaigns	Campaign sendings.
campaign_links	Trackable links (campaign_id, nonce, original_url).
campaign_stats	Open events.
campaign_link_stats	Click events.
Admin	
users	Admin users.
user_activity	Audit log.
settings	Platform-wide settings (key-value, JSON value).

5 Directory Structure

```

1 necoyoad/
2 |-- app/
3 |   |-- Models/                                # Eloquent models with morph traits
4 |   |   |-- Product.php
5 |   |   |-- Post.php                            # (also Page: same table, type
        discriminator)
6 |   |   |-- Category.php
7 |   |   |-- Banner.php
8 |   |   |-- Widget.php                          # Widget instance model
9 |   |   |-- WidgetRow.php
10 |   |   |-- WidgetColumn.php
11 |   |   |-- Store.php
12 |   |   |-- Description.php                    # Polymorphic description
13 |   |   |-- Property.php                       # Polymorphic EAV
14 |   |   |-- UrlAlias.php                       # Polymorphic SEO slug
15 |   |   |-- ...
16 |   |
17 |   |-- Filament/                              # Admin panel (Filament 3)
18 |   |   |-- Resources/
19 |   |   |   |-- ProductResource.php             # Declarative CRUD (form + table + pages)
20 |   |   |   |-- CategoryResource.php
21 |   |   |   |-- BannerResource.php
22 |   |   |   |-- PostResource.php
23 |   |   |   |-- MenuResource.php
24 |   |   |   |-- CampaignResource.php

```



```

25 | | | | -- ...
26 | | | | -- Pages/
27 | | | | -- Dashboard.php
28 | | | | -- Widgets/ # Dashboard widgets (Filament)
29 | | |
30 | | -- Livewire/ # Storefront reactive components
31 | | | | -- Storefront/
32 | | | | | -- ProductPage.php
33 | | | | | -- CartDrawer.php
34 | | | | | -- CheckoutForm.php
35 | | | | | -- SearchBar.php
36 | | | | -- WidgetEditor/ # Visual theme editor
37 | | | | | -- WidgetTree.php
38 | | | | | -- WidgetSettings.php
39 | | | | | -- DragDrop.php
40 | | |
41 | | -- View/ # Blade components
42 | | | | -- Components/
43 | | | | | -- Widgets/ # Widget module components
44 | | | | | | -- Banner.php # extends WidgetComponent
45 | | | | | | -- ProductList.php
46 | | | | | | -- CategoryList.php
47 | | | | | | -- ContactForm.php
48 | | | | | | -- RichText.php
49 | | | | | | -- Search.php
50 | | | | | | -- ...
51 | | | | | -- Sliders/ # Banner slider variants
52 | | | | | | -- NivoSlider.php
53 | | | | | | -- Slick.php
54 | | | | | | -- Camera.php
55 | | | | | | -- ...
56 | | | | | -- Layouts/
57 | | | | | | -- Storefront.blade.php
58 | | | | | | -- WidgetRow.blade.php
59 | | | | | | -- WidgetColumn.blade.php
60 | | | | | -- WidgetComponent.php # Base class for all widgets
61 | | | | -- Composers/
62 | | | | | -- WidgetComposer.php # The loadWidgets() equivalent
63 | | |
64 | | -- Services/ # Business logic
65 | | | | -- WidgetService.php # The NecoWidget equivalent
66 | | | | -- AssetManifest.php # The deps.php equivalent
67 | | | | -- StoreContext.php # Multi-store resolver
68 | | | | -- LanguageContext.php # Multi-language resolver
69 | | | | -- CampaignSender.php # Campaign send logic
70 | | | | -- TemplateResolver.php # Per-entity template override
71 | | |
72 | | -- Filters/ # The Hooks system (pipeline-based)
73 | | | | -- Filter.php # Facade
74 | | | | -- FilterPipeline.php
75 | | |
76 | | -- Traits/ # Reusable morph traits
77 | | | | -- HasDescriptions.php
78 | | | | -- HasProperties.php
79 | | | | -- HasStoreAssignment.php
80 | | | | -- HasSeoUrl.php
81 | | | | -- HasCategories.php
82 | | |
83 | | -- Jobs/ # Queue jobs
84 | | | | -- SendCampaignEmail.php
85 | | | | -- ProcessBounce.php
86 | | | | -- SendBirthdayEmail.php
87 | | |
88 | | -- Events/ # Laravel events

```

```

89 | |-- Listeners/
90 | |-- Mail/
91 | |-- Notifications/
92 | |-- Http/
93 | |   |-- Controllers/
94 | |   |-- Middleware/
95 | |   |-- Requests/           # Form validation (replaces $form_vars)
96 | |
97 | |-- Providers/
98 | |   |-- AppServiceProvider.php
99 | |   |-- WidgetServiceProvider.php # Registers all widgets + assets
100 | |   |-- HookServiceProvider.php  # Registers filter hooks
101 |
102 |-- database/
103 |   |-- migrations/           # 50+ migration files
104 |   |-- seeders/
105 |   |-- factories/
106 |
107 |-- resources/
108 |   |-- views/
109 |     |-- components/
110 |     |   |-- widgets/       # Widget Blade templates
111 |     |   |-- sliders/       # Slider Blade templates
112 |     |   |-- layouts/
113 |     |   |-- themes/         # Per-theme templates
114 |     |   |-- chironi/
115 |     |     |-- content/
116 |     |     |-- store/
117 |     |     |-- shared/
118 |     |     |-- banner/
119 |     |-- emails/             # Campaign + transactional emails
120 |
121 |   |-- css/                  # Tailwind CSS 4
122 |   |-- js/                   # Alpine.js + Vite entry
123 |   |-- vite-manifest.json    # The deps.php equivalent
124 |
125 |-- tests/                   # Pest 2 + Dusk
126 |-- docker/                  # Dockerfiles + compose
127 |-- .github/workflows/       # CI/CD

```

Listing 1: New Necoyoad directory structure (abridged).

6 Implementation Plan

6.1 Overview: 7 Phases, 80 Tasks, 8 Months

6.2 Phase 1: Foundation (Weeks 1–3)

1. Initialise Laravel 11 project with PHP 8.3.
2. Install Filament 3, Livewire 3, Alpine.js, Tailwind CSS 4.
3. Create database migrations (35 core tables, InnoDB, FKs).
4. Implement `Store` model + `StoreContext` middleware (domain/path/subdomain detection → `StoreContext::id()`).
5. Implement `Language` model + `LanguageContext` middleware (6-level detection).
6. Implement `Currency` model + switching.
7. Implement morph traits: `HasDescriptions`, `HasProperties`, `HasStoreAssignment`, `HasSeoUrl`, `HasCategories`.

Table 4: Implementation phases.

#	Phase	Tasks	Weeks	Deliverable
1	Foundation	12	3	Laravel project + DB schema + auth + multi-store/multi-language.
2	Widget Engine	15	4	WidgetService + WidgetComposer + WidgetComponent + 5 core widgets + visual editor.
3	Admin Panel	10	3	Filament resources for all entities + declarative CRUD.
4	Storefront	12	4	Catalog (product, category, search) + CMS (post, page) + checkout.
5	CMS + Banners	8	2	Posts/pages + menus + banner system with 5 slider templates.
6	Marketing	10	3	Campaign pipeline + tracking + bounce + birthday emails.
7	Polish + Deploy	13	3	Testing + Docker + CI/CD + documentation + launch.
Total		80	22	~5.5 months (1 senior dev); 3 months (2 devs).

8. Implement `Filter` facade (pipeline-based hooks).
9. Implement Laravel Events for observer notifications.
10. Set up Sanctum + Fortify (admin auth + API tokens).
11. Create base `NecoyoadResource` Filament class (breadcrumbs, activity logging, SEO tab, store tab).
12. Set up Docker + FrankenPHP development environment.

6.3 Phase 2: Widget Engine (Weeks 4–7) — The Critical Phase

1. Create `WidgetRow`, `WidgetColumn`, `Widget` models with JSON settings columns.
2. Implement `WidgetService::getTree()` — queries the widget tree with JSON path expressions (not LIKE).
3. Implement `WidgetService::getTree()` per-entity override (merges per-object widgets into default tree).
4. Implement `WidgetService::getTree()` with `only:` mode (skip default tree for embedded pages).
5. Implement `WidgetComposer` View Composer — calls `WidgetService`, populates `$widgets[$position]`.
6. Implement `WidgetComponent` base Blade component — auto-loads assets via `AssetManifest`, provides `data()` method.
7. Implement `AssetManifest` service — reads `vite-manifest.json`, loads matching CSS/JS for the current route + widget module.
8. Implement dynamic composition: page template iterates `$widgets` and renders `<x-dynamic-component>`.
9. Implement manual composition: `@stack` + `@push` for hardcoded widget positions.
10. Implement hybrid composition: both in the same template.
11. Create `WidgetServiceProvider` — registers all widget components + their asset mappings.
12. Create 5 core widget modules: `RichText`, `ProductList`, `CategoryList`, `ContactForm`, `Search`.
13. Implement `TemplateResolver` — 3-level resolution (entity template → config default → hardcoded fallback).

14. Implement visual editor hooks: `data-widget`, `data-position`, `data-row`, `data-column` attributes on every widget wrapper.
15. Implement `WidgetEditor` Livewire component (drag-and-drop widget tree, settings form, live preview).

6.4 Phase 3: Admin Panel (Weeks 8–10)

1. Create Filament Resources for: Product, Category, Manufacturer, Post, Page, Menu, Banner, Campaign, Contact, ContactList, Newsletter, Store, Language, Currency, User.
2. Each resource includes: form schema (with SEO tab, store assignment tab, descriptions tab per language), table columns, filters, actions.
3. Implement activity logging (Filament plugin).
4. Implement permission system (Filament shield + roles).
5. Implement file manager (Filament file upload + media library).
6. Implement theme/style editor (CSS rule overrides → `theme_styles` table → generated CSS file).
7. Implement widget layout manager (Filament page with Livewire drag-and-drop for widget tree editing).
8. Implement dashboard with charts (Filament widgets).
9. Implement settings page (per-store config).
10. Implement extension/module installer (register widget components dynamically).

6.5 Phase 4: Storefront (Weeks 11–14)

1. Implement product listing + detail (with per-entity widget composition + template override).
2. Implement category listing (tree, filters, pagination).
3. Implement search (Meilisearch or MySQL full-text).
4. Implement cart (Livewire reactive + session/Redis persistence).
5. Implement checkout (single-page Livewire: shipping → payment → confirm → success).
6. Implement customer auth (login, register, password reset, email verification).
7. Implement customer account (orders, addresses, profile, reviews, wallet balance).
8. Implement SEO URL rewriter (route model binding + slug column + `url_aliases` table).
9. Implement multi-store URL routing (domain → store detection → scoped queries).
10. Implement mobile theme switching (device detection → template swap, like original).
11. Implement asset loading (Vite + per-route CSS/JS + per-widget assets).
12. Implement page cache (Redis, per-store + per-language + per-device + per-customer-group).

6.6 Phase 5: CMS + Banners (Weeks 15–16)

1. Implement post/page controller (same table, `type` discriminator, per-entity widgets + template override).
2. Implement post category listing.

3. Implement menu system (menu + menu_link tree + polymorphic descriptions + EAV properties for per-link CSS/icon/submenu).
4. Implement links widget (recursive tree rendering, 3 submenu types: children, page_embed, html_content).
5. Implement banner system (jquery_plugin discriminator → dynamic Blade component).
6. Create 5 slider templates: NivoSlider, Slick, Camera, Fancybox gallery, CSS grid.
7. Implement per-item widget composition (widgets on individual banner slides, positioned with offsetX/offsetY).
8. Implement ntPlugins equivalent (Alpine.js store for slider config, \$dispatch for initialisation).

6.7 Phase 6: Marketing (Weeks 17–19)

1. Implement newsletter CRUD (HTML body editor).
2. Implement campaign CRUD (target contact lists, schedule, mailserver config).
3. Implement SendCampaignEmail queue job (per-recipient personalisation, link rewriting, tracking pixel append).
4. Implement tracking endpoints: /track/open/campaign/contact (writes campaign_stats), /track/click/nonce (writes campaign_link_stats, 302 redirect).
5. Implement bounce processing (ProcessBounce job, IMAP or webhook-based).
6. Implement unsubscribe mechanism (/unsubscribe/token route + List-Unsubscribe email header).
7. Implement birthday email job.
8. Implement promoter/recommended-products email job.
9. Implement campaign stats dashboard (opens, clicks, bounce rate, per-recipient detail).
10. Implement mailserver config (SMTP credentials, OAuth2 for Gmail, per-campaign mailserver selection).

6.8 Phase 7: Polish + Deploy (Weeks 20–22)

1. Write Pest 2 tests: unit (models, services), feature (HTTP endpoints, widget composition), integration (checkout, campaign send).
2. Write Dusk browser tests: visual editor drag-and-drop, checkout flow, search, multi-store switching.
3. Set up GitHub Actions CI: lint → test → build → deploy.
4. Create production Docker image (FrankenPHP + PHP 8.3 + Nginx).
5. Create docker-compose.yml for local dev (app + MySQL 8 + Redis + Meilisearch + Mailhog).
6. Write API documentation (Scribe or Laravel OpenAPI).
7. Write developer documentation: “Creating a New Widget Module” guide.
8. Write admin documentation: “Composing Pages with Widgets” guide. Security audit: CSRF, XSS, SQL injection, file upload, path traversal.
9. Performance audit: N+1 queries, cache hit rate, page load time. Seed demo data (products, posts, banners, menus, widgets). Deploy to staging. Launch.

7 The Widget Engine API Contract

The widget engine API contract — non-negotiable

Any implementation of the New Necoyoad widget engine **MUST** satisfy this contract:

Capability	Contract
Query widgets	<code>WidgetService::getTree(position, objectType?, objectId?)</code> returns nested rows → columns → widgets.
Register widget	A class extending <code>WidgetComponent</code> + a Blade template + a Filament settings form.
Inject data	<code>WidgetComponent::data()</code> method receives settings, returns data + template path.
Render	<code><x-dynamic-component></code> or <code>@stack/@push</code> .
Dynamic compose	View Composer populates <code>\$widgets[\$position]</code> before render.
Manual compose	Templates use <code>@stack</code> + <code>@push</code> to hardcode positions.
Hybrid compose	Both in the same template.
Per-entity override	<code>getTree()</code> accepts <code>objectType</code> + <code>objectId</code> ; merges per-object widgets.
Per-item composition only: mode	Same, with <code>objectType = 'banner_item'</code> . Skip default tree; load only per-object widgets (for embedded pages).
Asset loading	<code>AssetManifest</code> loads CSS/JS for current route + widget modules.
Visual editor hooks	<code>data-widget</code> , <code>data-position</code> , <code>data-row</code> , <code>data-column</code> on every wrapper.
Template override	Entity <code>template</code> column → config default → hardcoded fallback.
Device filtering	<code>show_in_mobile</code> , <code>show_in_tablet</code> , <code>show_in_desktop</code> in JSON settings.
Auth filtering	<code>customer_session_mode</code> : any, <code>logged_in</code> , <code>logged_out</code> .
Route filtering	<code>conditional_logic</code> : show/hide on specific routes.

8 Creating a New Widget Module

Developer guide: creating a new widget in New Necoyoad

Step 1: Create the component class

```

1 // app/View/Components/Widgets/FeaturedProducts.php
2
3 class FeaturedProducts extends WidgetComponent
4 {
5     public function data(): array
6     {
7         $products = Product::with('descriptions')
8             ->where('featured', true)
9             ->forStore(StoreContext::id())
10            ->forLanguage(LanguageContext::id())
11            ->limit($this->settings['limit'] ?? 5)
12            ->get();
13

```

```

14         return [
15             'products' => $products,
16             'settings' => $this->settings,
17         ];
18     }
19 }

```

Step 2: Create the Blade template

```

1  {{-- resources/views/components/widgets/featured-products.blade.php --}}
2  @php $tpl = theme_path('shared') ?? 'choroni'; @endphp
3  <li id="{{ $widgetName }}" class="widget featured-products nt-editable"
4      data-widget="{{ $widgetName }}" data-position="{{ $position }}">
5      @if ($headingTitle ?? '')
6          <h3>{{ $headingTitle }}</h3>
7      @endif
8      <div class="product-grid">
9          @foreach ($products as $product)
10             <div class="product-card">
11                 title }}" />
13                 <a href="{{ route('store.product', $product) }}">{{
14                     $product->title }}</a>
15                 <span class="price">{{ format_currency($product->price)
16                     }}</span>
17             </div>
18         @endforeach
19     </div>
20 </li>

```

Step 3: Register the widget + assets

```

1  // app/Providers/WidgetServiceProvider.php
2
3  public function boot(): void
4  {
5      AssetManifest::registerWidget('featured-products', [
6          'css' => ['resources/css/widgets/featured-products.css'],
7          'js'  => ['resources/js/widgets/featured-products.js'],
8          'routes' => ['*', 'store/product', 'common/home'],
9      ]);
10 }

```

Step 4: Create the Filament settings form (for the admin)

```

1  // In the widget layout manager
2  Forms\Components\Section::make('Featured Products')
3      ->schema([
4          Forms\Components\TextInput::make('limit')
5              ->label('Number of products')
6              ->default(5),
7          Forms\Components\TextInput::make('heading_title')
8              ->label('Heading title'),
9      ]);

```

Done. The admin can now drag the “Featured Products” widget into any position on any page. The widget renders the products, loads its CSS/JS, and emits the `nt-editable` attributes for the visual editor.

9 What is Discarded

Table 5: What is discarded from the original.

Discarded	Replaced by
MyISAM tables	InnoDB with FKs + transactions.
Double-MD5 passwords	bcrypt / argon2id.
LIKE '%key=value%' queries	JSON column path expressions.
Raw SQL (Model::buildSQLQuery)	Eloquent + query builder.
PHPMailer 5.0	Laravel Mail (Symfony Mailer).
validateTokens() stub	Laravel Sanctum.
.htaccess rewrite rules	Laravel routing + middleware.
map.php 843-line switch	Laravel autoloading + service providers.
Hand-rolled session handler	Laravel session (Redis driver).
File-based cache	Redis.
{%widget_name%} string substitution	Blade dynamic components / @stack.
Serialised PHP settings blobs	JSON columns.
No testing	Pest 2 + Dusk.
No unsubscribe	Built-in unsubscribe + List-Unsubscribe header.
Vestigial bounce processor	Working bounce processing via IMAP/webhook.

10 Milestones

Table 6: Project milestones.

#	Week	Milestone	Acceptance criteria
M1	3	Foundation	Multi-store + multi-language working; auth; DB schema; Docker dev env.
M2	7	Widget Engine	5 core widgets; visual editor; dynamic + manual + hybrid composition; per-entity overrides.
M3	10	Admin Panel	All Filament resources; widget layout manager; theme editor; file manager.
M4	14	Storefront	Product listing + detail; cart; checkout; search; SEO URLs; multi-store routing.
M5	16	CMS + Banners	Posts/pages; menus; banner system with 5 slider templates; per-item widgets.
M6	19	Marketing	Campaign send; tracking; bounce; unsubscribe; birthday emails.
M7	22	Launch	Tests pass; CI/CD; Docker; documentation; demo data; staging deployed.

11 Conclusion

11.1 The Vision

New Necoyoad is a digital web agency platform that preserves the original’s most valuable asset — the widget engine — while discarding its technical debt. It is built on Laravel 11, PHP 8.3, Filament 3, Livewire 3, and MySQL 8 InnoDB. It supports multi-store, multi-language, per-entity widget overrides, per-entity template overrides, a visual theme editor, a self-hosted marketing pipeline, and a plugin-aware banner system — all on a modern, testable, secure foundation.

11.2 The Widget Engine Survives

The widget engine — the platform’s soul — survives the rebuild. The seven core abstractions are preserved, each with a concrete Laravel equivalent. The developer experience (“create a compo-

nent + template + settings form + register assets”) is the same. The admin experience (“drag a widget into a position, configure it, see it on the page”) is the same. The three composition modes (dynamic, manual, hybrid) are the same.

11.3 The Timeline is Achievable

22 weeks (5.5 months) for a single senior developer; 3 months for a team of two. The widget engine phase (Phase 2) is the critical path — it must be right before anything else can be built on top of it.

11.4 Final Word

The original Necoyoad was a characterful, ambitious platform built with care and a strong vision. The New Necoyoad preserves that vision — the widget-driven page builder, the polymorphic object spine, the multi-store architecture, the marketing pipeline — while leaving behind the technical debt of a 2009-era codebase. The result is a platform that is both familiar (same concepts, same developer experience) and modern (Laravel 11, PHP 8.3, testing, security, Docker).

The blueprint is complete. The build can begin.